

VERSÃO DO ALUNO

# LÓGICA DE PROGRAMAÇÃO

Aprendendo a pensar como um programador

## AUTORES

Mateus Pereira Sousa

RU: 5155577

Thais Moura de Barros

RU: 4921704

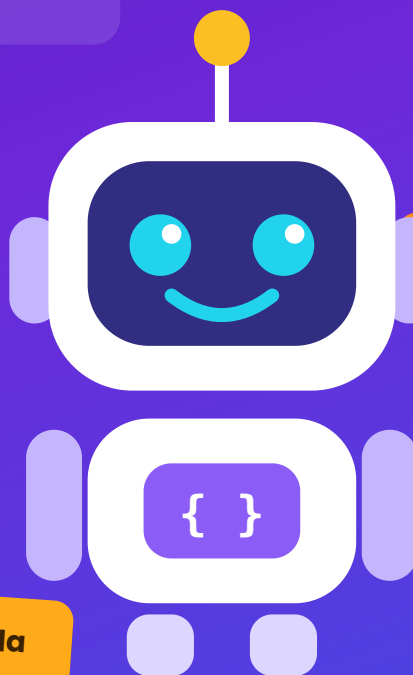
QUANDO BANDEIRA VERDE CLICADA

mova 10 passos

REPITA 5 VEZES

mudar pontos por 1

SE personagem tocar estrela



PÚBLICO  
adolescentes de 11 a 14 anos



MODALIDADE  
aulas práticas e teóricas



FERRAMENTA PRINCIPAL  
Scratch



CARGA SUGERIDA  
20 a 25 aulas de 1h30 a 2h



NÍVEL  
iniciante



# SUMÁRIO

## 1 Módulo 1 — Pensamento Computacional 4

|   |                           |    |
|---|---------------------------|----|
| 1 | O que é programação?      | 5  |
| 2 | Pensamento lógico         | 7  |
| 3 | Algoritmos                | 8  |
| 4 | Decomposição de problemas | 9  |
| 5 | Padrões                   | 10 |

## 2 Módulo 2 — Primeiros conceitos de programação 11

|   |                     |    |
|---|---------------------|----|
| 6 | Dados e informações | 12 |
| 7 | Variáveis           | 14 |
| 8 | Operadores          | 16 |
| 9 | Entrada e saída     | 18 |

## 3 Módulo 3 — Tomando decisões 19

|    |                     |    |
|----|---------------------|----|
| 10 | Condicionais        | 20 |
| 11 | Operadores lógicos  | 21 |
| 12 | Condições aninhadas | 22 |

## 4 Módulo 4 — Repetição 23

|    |                         |    |
|----|-------------------------|----|
| 13 | O que são loops?        | 24 |
| 14 | Repetições com condição | 25 |
| 15 | Contadores              | 26 |

## 5 Módulo 5 — Organização do código 27

|    |                           |    |
|----|---------------------------|----|
| 16 | Funções e procedimentos   | 28 |
| 17 | Listas                    | 29 |
| 18 | Organização de algoritmos | 30 |

## 6 Módulo 6 — Encontrando problemas 31

|    |           |    |
|----|-----------|----|
| 19 | Bugs      | 32 |
| 20 | Debugging | 33 |
| 21 | Testes    | 34 |

## 7 Módulo 7 — Programação visual com Scratch 35

|    |                       |    |
|----|-----------------------|----|
| 22 | Conhecendo o Scratch  | 36 |
| 23 | Movimento e eventos   | 37 |
| 24 | Condições e loops     | 38 |
| 25 | Variáveis e pontuação | 39 |

## 8 Módulo 8 — Projetos 40

|    |                                 |    |
|----|---------------------------------|----|
| 26 | Projeto 1 — Quiz                | 41 |
| 27 | Projeto 2 — Jogo de coleta      | 42 |
| 28 | Projeto 3 — Aventura interativa | 43 |
| 29 | Projeto final                   | 44 |



# COMO UTILIZAR ESTA APOSTILA

Cada aula possui cinco momentos:



## 1. Aprender

Apresentação do conceito.



## 2. Observar

Exemplo prático.



## 3. Praticar

Exercícios guiados.



## 4. Desafio

Problema para o aluno resolver.



## 5. Criar

Aplicação do conhecimento em um pequeno projeto.

### AUTORES DESTE MATERIAL

Mateus Pereira Sousa — RU: 5155577

Thais Moura de Barros — RU: 4921704

## MÓDULO 1

# PENSAMENTO COMPUTACIONAL

### NESTE MÓDULO

- 1 O que é programação?
- 2 Pensamento lógico
- 3 Algoritmos
- 4 Decomposição de problemas
- 5 Padrões



## Objetivos

Ao final da aula, o aluno deverá compreender:

- O que é programação;
- O que é um programa;
- Para que servem os algoritmos;
- Que computadores executam instruções.



## Aprendendo

Programação é o processo de criar instruções para que um computador realize determinada tarefa.

Um programa pode:

- mostrar uma mensagem;
- tocar uma música;
- calcular uma conta;
- controlar um personagem;
- armazenar informações;
- executar um jogo.

Quando programamos, precisamos dizer ao computador **o que fazer e como fazer**.



## Pense

Imagine que você precisa ensinar um robô a fazer um sanduíche.

Você poderia dizer:

*"Faça um sanduíche."*

Mas o robô talvez não saiba o que isso significa.

Precisamos dividir a tarefa:

- 1 Pegar o pão.
- 2 Colocar o pão em um prato.
- 3 Pegar o queijo.
- 4 Colocar o queijo no pão.

5 Pegar o outro pedaço de pão.

6 Colocar por cima.

Essa sequência é um **algoritmo**.



## Exercício

Escreva as instruções para:

**A Escovar os dentes**

**B Fazer um suco**

**C Abrir um jogo no computador**



## Desafio

Seu robô precisa atravessar uma sala.

Ele só entende:

- ANDAR;
- VIRAR À DIREITA;
- VIRAR À ESQUERDA.

Crie uma sequência de comandos para levá-lo até uma determinada posição.



## Aprendendo

Pensamento lógico é a capacidade de analisar uma situação e encontrar uma sequência coerente para chegar a uma solução.

Programadores constantemente fazem perguntas como:

*Qual é o problema?*

*Quais informações eu tenho?*

*O que preciso descobrir?*

*Quais passos devo executar?*



## Atividade — Qual vem depois?

Complete:

### Sequência 1

2

4

6

8

### Sequência 2

5

10

15

20

### Sequência 3

1

2

4

8

### Sequência 4



## Desafio

Crie uma sequência própria e peça para outro aluno descobrir a regra.



### Aprendendo

Um algoritmo é uma sequência organizada de instruções para resolver um problema.

Um algoritmo precisa ser:

- organizado;
- compreensível;
- finito;
- capaz de chegar a um resultado.



### Exercício

Coloque os passos abaixo na ordem correta:

- Colocar o cadarço no tênis.
- Pegar o tênis.
- Amarrar o cadarço.
- Colocar o tênis no pé.
- Pegar o cadarço.



### Desafio

Crie um algoritmo para:

**"Preparar-se para ir à escola."**

O aluno deverá utilizar pelo menos 8 passos.





## Aprendendo

Problemas grandes podem parecer difíceis porque possuem muitas partes.

Uma estratégia utilizada pelos programadores é a **decomposição**.

Isso significa:

*Dividir um problema grande em problemas menores.*

### Exemplo

Problema:

**Criar um jogo de futebol.**

Podemos dividir em:

- Criar jogador;
- Criar bola;
- Fazer o jogador andar;
- Fazer a bola se movimentar;
- Criar gol;
- Criar placar;
- Criar regras;
- Criar vitória.



## Exercício

Divida o problema:

**"Criar um aplicativo de lista de tarefas."**

em pelo menos cinco partes.



## Aprendendo

Programadores procuram padrões para descobrir regras e evitar trabalho repetitivo.

Exemplo:



Regra:

Somar 10.



## Desafio

Descubra a regra:

A

3

6

9

12

B

100

90

80

70

C

1

3

5

7

D

2

4

8

16



## MÓDULO 2

# PRIMEIROS CONCEITOS DE PROGRAMAÇÃO

### NESTE MÓDULO

- 6 **Dados e informações**
- 7 **Variáveis**
- 8 **Operadores**
- 9 **Entrada e saída**



## Aprendendo

Programas trabalham com informações.

Alguns exemplos:



```
Nome = "Lucas"  
Idade = 13  
Altura = 1.62  
Aluno = verdadeiro
```

Podemos ter diferentes tipos de informação:

### Texto



```
"Olá!"
```

### Número



```
15
```

### Verdadeiro ou falso



```
verdadeiro  
falso
```



## Exercício

Classifique:

| Informação    | Tipo |
|---------------|------|
| "Maria"       | ?    |
| 15            | ?    |
| 3.14          | ?    |
| verdadeiro    | ?    |
| "Programação" | ?    |

# 7

## CAPÍTULO 7 VARIÁVEIS



### Aprendendo

Uma variável pode ser imaginada como uma **caixa que guarda uma informação**.

Por exemplo:



```
idade = 12
```

A variável chamada `idade` guarda o número 12.

Outro exemplo:



```
nome = "Ana"
```



### Atividade

Crie uma ficha de personagem:



```
Nome:  
Idade:  
Pontos:  
Vida:  
Nível:
```

Transforme cada item em uma variável.



## Desafio

Crie um personagem de videogame com:

- nome;
- vida;
- moedas;
- nível;
- força.

Depois imagine que ele ganhou 20 moedas.

O que acontece com a variável `moedas` ?



## Aprendendo

Operadores permitem realizar operações.

### Matemáticos

```
+ soma  
- subtração  
* multiplicação  
/ divisão
```

### Comparação

```
> maior  
< menor  
>= maior ou igual  
<= menor ou igual  
= igual
```



## Exercícios

1

Quanto é:

```
10 + 5
```

2

Quanto é:

```
20 - 8
```



3

Quanto é:

  
`6 × 4`

4

João tem 15 anos e Maria tem 12.

A afirmação abaixo é verdadeira?

  
`João > Maria`



## Aprendendo

Um programa pode:

### Receber informações

Exemplo:

*Qual é seu nome?*

### Processar informações

Exemplo:

*Descobrir a idade.*

### Mostrar informações

Exemplo:

*"Você tem 13 anos."*

Esse processo pode ser representado como:

ENTRADA



PROCESSAMENTO



SAÍDA



## Desafio

Crie um algoritmo que:

- 1 Pergunte o nome.
- 2 Pergunte a idade.
- 3 Mostre uma mensagem personalizada.



## MÓDULO 3

# TOMANDO DECISÕES

### NESTE MÓDULO

10

**Condicionais**

---

11

**Operadores lógicos**

---

12

**Condições aninhadas**



## Aprendendo

Programas precisam tomar decisões.

Imagine:

*Se estiver chovendo, levar guarda-chuva.*

Podemos escrever:



```
SE estiver_chovendo  
    levar guarda-chuva
```

Outro exemplo:



```
SE idade >= 12  
    mostrar "Pode participar"  
SENÃO  
    mostrar "Ainda não pode"
```



## Exercício

Complete:



```
SE temperatura > 30
```

```
    _____  
SENÃO
```

```
    _____
```



## Aprendendo

Às vezes precisamos verificar mais de uma condição.

**E**

As duas condições precisam ser verdadeiras.



```
idade >= 12 E possui_ingresso
```

**OU**

Pelo menos uma precisa ser verdadeira.



```
sábado OU domingo
```

**NÃO**

Inverte o valor.



```
NÃO está_chovendo
```



## Desafio

Para entrar em um campeonato, o jogador precisa:

- ter pelo menos 12 anos;
- possuir autorização.

Crie a condição.



## Aprendendo

Uma condição pode conter outra condição.

Exemplo:

```
SE possui_ingresso
  SE idade >= 12
    entrar
  SENÃO
    mostrar "Idade insuficiente"
SENÃO
  mostrar "Compre um ingresso"
```



## Atividade

Crie um sistema para uma montanha-russa.

Regras:

- idade mínima: 12 anos;
- altura mínima: 1,40 m.

O aluno deverá criar as condições.



## MÓDULO 4

# REPETIÇÕES

### NESTE MÓDULO

- 13 O que são loops?
- 14 Repetições com condição
- 15 Contadores



## Aprendendo

Imagine que queremos mostrar:

```
Olá!  
Olá!  
Olá!  
Olá!  
Olá!
```

Podemos escrever cinco comandos.

Mas é mais inteligente dizer:

```
REPITA 5 VEZES  
  mostrar "Olá!"
```

Isso é uma **repetição** ou **loop**.



## Desafio

Crie um algoritmo que mostre:

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

utilizando uma repetição.





## Aprendendo

Algumas repetições continuam enquanto uma condição for verdadeira.

Exemplo:



```
ENQUANTO vida > 0  
  jogar
```

O jogo continuará enquanto o personagem tiver vida.



## Atividade

Imagine um personagem com:



```
vida = 100
```

A cada ataque ele perde 20 pontos.

Quantos ataques são necessários para chegar a 0?



### Aprendendo

Contadores são variáveis utilizadas para contar alguma coisa.

Exemplo:



```
pontos = 0
```

```
quando ganhar moeda:
```

```
    pontos = pontos + 1
```



### Desafio

Crie um sistema de pontuação:

- moeda = +1 ponto;
- inimigo = +5 pontos;
- perder vida = -1 ponto.

O aluno deverá criar as variáveis e regras.



## MÓDULO 5

# ORGANIZAÇÃO DO CÓDIGO

### NESTE MÓDULO

- 16 Funções e procedimentos
- 17 Listas
- 18 Organização de algoritmos



## Aprendendo

Imagine que temos uma ação que usamos várias vezes.

Em vez de escrever tudo novamente, podemos criar uma função.

Exemplo:

```
FUNÇÃO pular  
    subir  
    descer  
FIM
```

Depois:

```
pular()  
pular()  
pular()
```



## Desafio

Crie funções para:

- atacar;
- pular;
- correr;
- receber dano.

Depois imagine um jogo utilizando essas funções.



## Aprendendo

Uma lista guarda vários valores.

Exemplo:

```
frutas:  
maçã  
banana  
laranja  
uva
```

No Scratch, podemos criar uma lista chamada:

**Frutas**

e adicionar vários itens.



## Atividade

Crie uma lista chamada:

**Inventário**

Adicione:

- espada;
- poção;
- escudo;
- chave.

Depois crie uma ação para adicionar um novo item.



## Aprendendo

Um bom programa deve ser organizado.

Podemos utilizar:

- nomes claros;
- comentários;
- funções;
- blocos separados;
- variáveis bem nomeadas.

Em vez de:

```
x = 10  
y = 5  
z = x + y
```

é mais fácil entender:

```
pontos = 10  
bonus = 5  
pontuacao_final = pontos + bonus
```



## Desafio

Peça aos alunos para melhorar um algoritmo desorganizado fornecido pelo professor.



## MÓDULO 6

# BUGS E DEBUG

### NESTE MÓDULO

19

**Bugs**

20

**Debugging**

21

**Testes**



## Aprendendo

Um **bug** é um problema no programa.

Exemplo:

Queremos:



```
2 + 2 = 4
```

Mas o programa mostra:



```
2 + 2 = 5
```

Existe um problema.



## Atividade — Detetive de Bugs

Apresente:



```
idade = 12
```

```
SE idade < 12
```

```
    mostrar "Pode participar"
```

```
SENÃO
```

```
    mostrar "Não pode participar"
```

Pergunte:

*Qual é o problema?*





## Aprendendo

Debugging significa investigar e corrigir problemas no programa.

Podemos seguir quatro passos:

### Reproduzir

Fazer o problema acontecer novamente.

1

### Observar

Descobrir o que aconteceu.

2

### Investigar

Encontrar onde está o problema.

3

### Corrigir e testar

Fazer uma alteração e verificar novamente.

4



## Desafio

O professor entrega um projeto Scratch com três bugs.

Os alunos devem:

- 1 Encontrar;
- 2 Explicar;
- 3 Corrigir;
- 4 Testar.



## Aprendendo

Não basta criar um programa.

Precisamos verificar se ele funciona.

Imagine um programa que calcula idade.

Teste:



```
Entrada: 10  
Entrada: 12  
Entrada: 18  
Entrada: 100  
Entrada: 0
```

Podemos descobrir problemas que não apareceram no primeiro teste.



## Atividade

Crie uma tabela:

| Entrada | Resultado esperado | Resultado obtido |
|---------|--------------------|------------------|
| 10      | Criança            |                  |
| 12      | Adolescente        |                  |
| 18      | Adulto             |                  |

## MÓDULO 7

# SCRATCH

### NESTE MÓDULO

**22** Conhecendo o Scratch

---

**23** Movimento e eventos

---

**24** Condições e loops

---

**25** Variáveis e pontuação



## Objetivo

Apresentar:

- palco;
- personagens;
- blocos;
- scripts;
- eventos;
- movimento;
- aparência;
- som;
- controle;
- sensores;
- operadores;
- variáveis.



## Primeira atividade

Criar um personagem que:

- 1 Aparece na tela;
- 2 Diz "Olá!";
- 3 Anda;
- 4 Muda de direção;
- 5 Toca um som.



### Conceito

Um programa precisa saber **quando começar uma ação**.

No Scratch podemos usar:

*Quando a bandeira verde for clicada.*

Depois:



`mova 10 passos`



### Desafio

Faça o personagem:

- andar;
- virar;
- dizer uma mensagem;
- tocar um som.

Depois faça uma animação utilizando pelo menos dois personagens.



## Projeto rápido — "Pegue a estrela"

## Objetivo

O personagem precisa pegar uma estrela.

## Elementos

- personagem;
- estrela;
- cenário;
- movimento;
- colisão;
- pontuação.

## Lógica



## QUANDO BANDEIRA VERDE CLICADA

```
definir pontos = 0
```

```
SE personagem tocar estrela  
  mudar pontos por 1  
  esconder estrela
```

Depois:



## PARA SEMPRE

```
SE personagem tocar estrela  
  mudar pontos por 1  
  mover estrela para posição aleatória
```

**Mini projeto — "Pegue as moedas"****Objetivo**

O jogador precisa coletar moedas.

**Regras**

Cada moeda vale 1 ponto.

Quando o jogador chegar a 10 pontos:

*"Você venceu!"*

**Conceitos utilizados**

- variável;
- repetição;
- condição;
- eventos;
- pontuação.



## MÓDULO 8

# PROJETOS

### NESTE MÓDULO

- 26 **Projeto 1 — Quiz**
- 27 **Projeto 2 — Jogo de coleta**
- 28 **Projeto 3 — Aventura interativa**
- 29 **Projeto final**





## PROJETO 1 QUIZ



### Objetivo

Criar um jogo de perguntas e respostas.

#### O jogo deverá possuir:

- pelo menos 5 perguntas;
- sistema de pontuação;
- respostas corretas e incorretas;
- mensagem final.

#### Exemplo



Qual é a capital do Brasil?

- A) São Paulo
- B) Brasília
- C) Rio de Janeiro

Se a resposta for correta:



+1 ponto

Se estiver errada:



0 pontos

#### Desafio extra

Criar diferentes níveis:

- Fácil;
- Médio;
- Difícil.



PROJETO 2

## JOGO DE COLETA



### Objetivo

Criar um jogo em que o personagem precisa coletar objetos.

#### Requisitos

- personagem;
- cenário;
- objeto coletável;
- pontuação;
- cronômetro;
- condição de vitória.

#### Exemplo

O jogador possui 30 segundos para coletar 10 moedas.

#### Desafios extras

Adicionar:

- inimigos;
- vidas;
- fases;
- sons;
- power-ups.



PROJETO 3

## AVENTURA INTERATIVA



### Objetivo

Criar uma história em que as escolhas do jogador alteram o resultado.

#### Exemplo

Você está em uma floresta.

Você deseja:

- 1 – Seguir pela esquerda
- 2 – Seguir pela direita

Se escolher esquerda:

*Encontrou uma ponte.*

Se escolher direita:

*Encontrou uma caverna.*

Cada caminho pode levar a um final diferente.



### Conceitos utilizados

- entrada;
- condições;
- variáveis;
- eventos;
- histórias;
- decisões.

## PROJETO FINAL



# CRIE SEU PRÓPRIO JOGO

Agora o aluno deverá criar um projeto autoral.

Pode ser:



**Jogo de aventura**



**Corrida**



**Futebol**



**Jogo com animais**



**RPG**



**Jogo espacial**



**Puzzle**



**Quiz**



**Plataforma**



**Jogo de pontuação**

## REQUISITOS DO PROJETO FINAL

O projeto deverá possuir:

- |                                                                               |                                                                           |
|-------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <input type="checkbox"/> Pelo menos 1 personagem principal.                   | <input type="checkbox"/> Pelo menos 1 condição.                           |
| <input type="checkbox"/> Pelo menos 1 cenário.                                | <input type="checkbox"/> Pelo menos 1 repetição.                          |
| <input type="checkbox"/> Movimento ou interação.                              | <input type="checkbox"/> Pelo menos 1 evento.                             |
| <input type="checkbox"/> Pelo menos 1 variável.                               | <input type="checkbox"/> Pelo menos 1 objetivo.                           |
| <input type="checkbox"/> Sistema de pontuação ou outra informação armazenada. | <input type="checkbox"/> Uma condição de vitória ou finalização.          |
|                                                                               | <input type="checkbox"/> Pelo menos 1 elemento criado pelo próprio aluno. |



## PLANEJAMENTO DO PROJETO

Antes de programar, o aluno deverá responder:

**1 Qual será o nome do jogo?**

---

---

**2 Qual é o objetivo?**

---

---

**3 Quem é o personagem principal?**

---

---

**4 O que o jogador precisa fazer?**

---

---

**5 Como o jogador ganha?**

---

---

**6 Como o jogador perde?**

---

---

**7 Quais variáveis serão utilizadas?**

---

---

**8 Quais serão as regras?**

---

---

**9 Quantas fases haverá?**

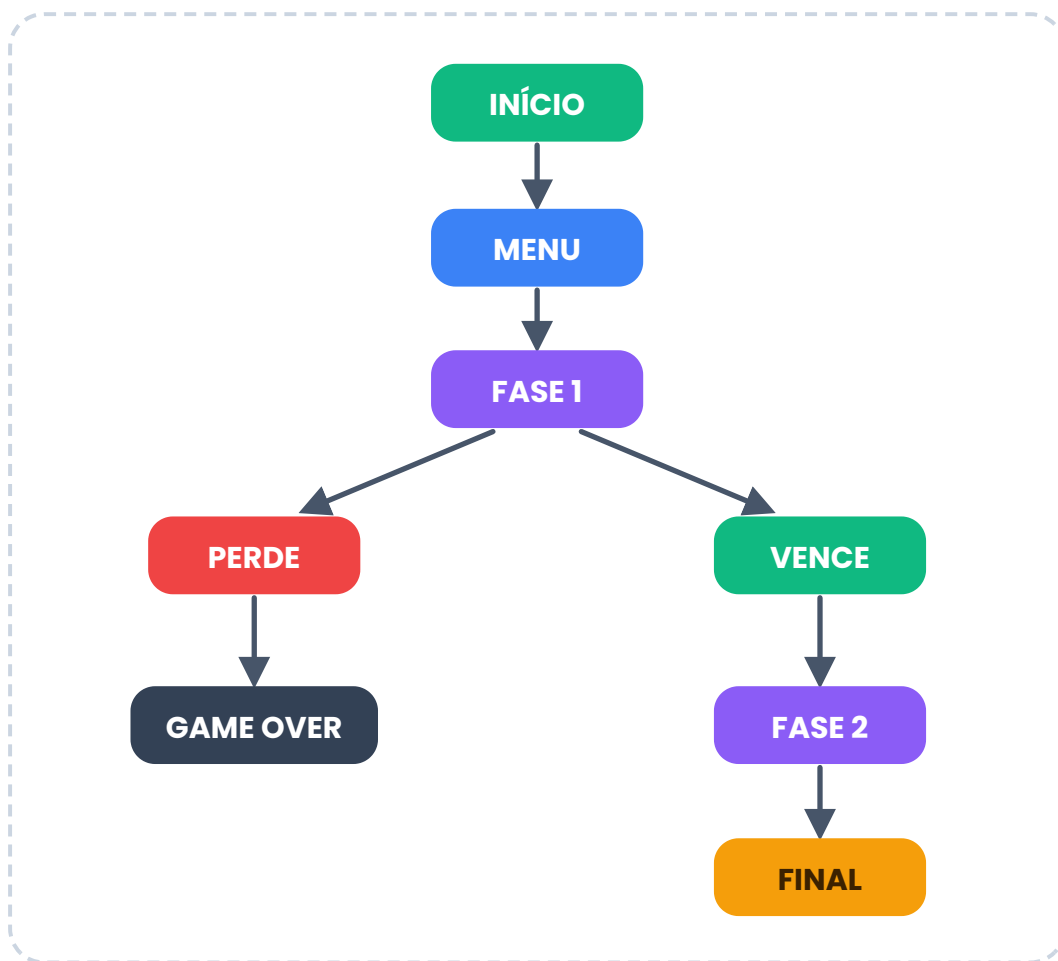
---

---



## MAPA DO JOGO

O aluno deverá desenhar:





## TRABALHO EM DUPLAS

Algumas atividades podem ser realizadas em duplas.

Uma estratégia interessante é utilizar dois papéis:



### Programador

Responsável por escrever o código.



### Investigador

Responsável por observar, testar e procurar erros.

Depois de alguns minutos, os papéis são trocados.

Isso ajuda a desenvolver:

- comunicação;
- colaboração;
- raciocínio;
- capacidade de explicar soluções.



## CERTIFICADO / CONCLUSÃO

Ao finalizar o projeto, o aluno poderá apresentar seu jogo para os colegas.

Sugestão de encerramento:



### "Feira de Projetos"

Cada aluno ou dupla apresenta:

- 1 Nome do projeto;
- 2 Objetivo;
- 3 Como jogar;
- 4 Conceitos utilizados;
- 5 Maior dificuldade encontrada;
- 6 Como resolveu o problema;
- 7 O que gostaria de melhorar.



## DESAFIOS EXTRAS PARA ALUNOS AVANÇADOS

Como alunos de 11 a 14 anos podem apresentar níveis muito diferentes, alguns desafios podem ficar disponíveis para quem terminar antes.

1

### Desafio 1

Adicionar vidas ao jogo.

2

### Desafio 2

Criar diferentes fases.

3

### Desafio 3

Adicionar cronômetro.

4

### Desafio 4

Criar ranking de pontuação.

5

### Desafio 5

Adicionar inimigos.

6

### Desafio 6

Criar itens especiais.

7

### Desafio 7

Criar diferentes finais.

8

### Desafio 8

Adicionar efeitos sonoros.

9

### Desafio 9

Criar um menu inicial.

10

### Desafio 10

Permitir que o jogador escolha a dificuldade.





## MENSAGEM FINAL AO ALUNO

**Programar não significa apenas escrever código.**

Programar significa **aprender a resolver problemas.**

Quando um programa não funciona, você não fracassou.

Você encontrou um problema que ainda precisa ser investigado.

Quando você divide um problema grande em partes menores, está pensando como um programador.

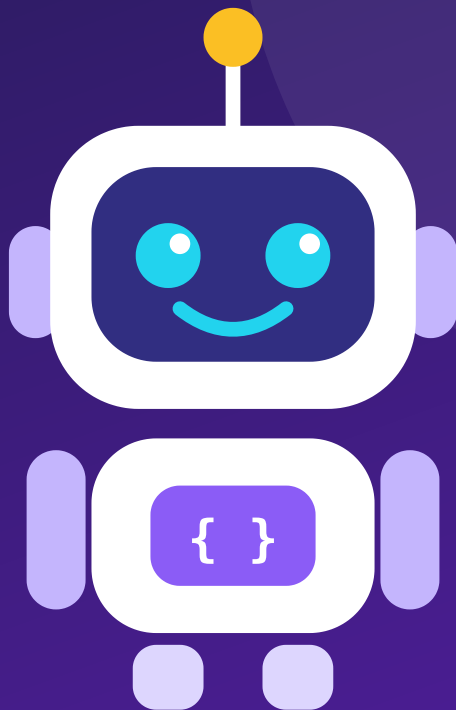
Quando testa uma solução, está pensando como um programador.

Quando encontra um erro e tenta descobrir por que ele aconteceu, está pensando como um programador.

E quando cria algo que antes não existia...

---

**você está programando.** 🚀



ENCERRAMENTO DO MATERIAL

## Projeto de Extensão — Introdução à Programação

### PÚBLICO

11 a 14 anos

### FERRAMENTA

Scratch

### COMPETÊNCIAS TRABALHADAS

pensamento computacional, lógica, resolução de problemas, criatividade, colaboração, programação e debugging.